

AnyShare / Hookinfo.ini 对话导出

主题：AnyShare SyncClientService、sharetool.exe HOOK、Git 命令挂起、Hookinfo.ini 排除配置与 PowerShell 修改脚本

用户

anyshare(<https://www.aishu.cn/cn/intelligent-cloud-storage>) 这个软件在win10 系统下会有一个 SyncClientService 服务，这个服务的执行文件路径为 C:\Program Files(x86)\AISHU\Sync\SyncDisk.exe，这个服务看起来会监控进程，因为有进程启动时会有一个 `C:\Program Files(x86)\AISHU\Sync\sharetool.exe --hooks 3 <pid>` 程序启动，我有个自己成 rust 程序需要依赖执行 git 命令统计 ai 代码行数，这个程序在虚拟机下我执行 git 命令的时候会偶发的在后台挂起，有时候在终端单独执行 git 命令也会挂起，我于是把服务停止掉就不会挂起，执行 git 命令后用 process monitor 看到 sharetool.exe 会创建 `Thread Create` 几个线程，但是只有一个线程会 `Thread Exit`，在不卸载 anyshare 的情况有什么解决方案

助手

结论：**优先改 AnyShare 的 HOOK 配置，不要只在 Rust 里绕。**你看到的 `sharetool.exe --hooks <pid>` 基本就是 AnyShare 同步盘的注入逻辑。爱数官方博客说明，同步盘会把 `winhook.dll / winhook64.dll` 注入进程，并由 `sharetool` 把 hook DLL 注入到应用程序中；默认“最大化注入”会注入所有能注入的进程，官方也明确说遇到软件卡死、无法启动时可加入“不注入名单”。Windows 的全局 Hook 本身也可能把 DLL 加载到其他进程上下文中运行。

最推荐方案：把 Git 和你的 Rust 程序加入 AnyShare 的 Exclude

先找到 `Hookinfo.ini`。官方说这个配置文件在安装目录下，和 `tray.exe` 同级。你可以用管理员 CMD 查一下：

```
dir "C:\Program Files (x86)\AISHU" /s /b | findstr /i Hookinfo.ini
```

备份后编辑：

```
copy "C:\Program Files (x86)\AISHU\Sync\Hookinfo.ini" "C:\Program Files (x86)\AISHU\Sync\Hookinfo.ini.bak"
notepad "C:\Program Files (x86)\AISHU\Sync\Hookinfo.ini"
```

在 `Hookinfo.ini` 里加或合并下面内容。注意：如果原来已有 `[Exclude]`，不要重复新建段，继续往后排 `Path1/Path2`、`Process1/Process2`。

```
[Global]
Mode=0

[Exclude]
; Git for Windows 整个安装目录都不注入，覆盖 git.exe、ssh.exe、git-remote-https.exe 等子程序
Path0=C:\Program Files\Git\

; 你的 Rust 程序所在目录
Path1=C:\your\rust\tool\dir\

; 保险起见，也按进程名排除
Process0=git.exe
Process1=ssh.exe
Process2=git-remote-https.exe
Process3=git-credential-manager.exe
Process4=git-lfs.exe
Process5=your_tool.exe
```

然后重启 AnyShare 服务和托盘程序：

```
sc stop SyncClientService
sc start SyncClientService
```

也可以直接在“服务”里重启 `SyncClientService`，并退出/重新打开 AnyShare 客户端。

验证方式：再次执行你的 Rust 程序或单独执行 `git status`、`git diff`，用 ProcMon 过滤：

```
Process Name is sharetool.exe
Command Line contains --hooks
Command Line contains git
Command Line contains your_tool.exe
```

理想结果是：你的程序、`git.exe`、Git 子进程启动时，不再触发

`sharetool.exe --hooks <pid>`。

如果 Exclude 不稳定：改成最小化注入

爱数官方说明 `Mode=1` 是“最小化注入”，只注入必须的少数进程；还明确说“大面积进程卡死”时可以考虑设置为 `1` 来验证是不是 HOOK 导致。

```
[Global]
Mode=1
```

这个方案副作用更大：AnyShare 同步盘的“打开云端占位文件时先下载再打开”等能力，可能对部分应用不再生效，因为同步盘的一个核心实现就是在系统启动应用前拦截/HOOK，然后触发下载。所以我建议先用 `[Exclude]`，不行再用 `Mode=1`。

你的 Rust 程序里也建议做防挂处理

虽然根因更像 AnyShare 注入，但你的工具执行 Git 时也建议这样写，避免 pager、交互认证、外部 diff 等造成假性挂起。Git 官方文档说明 `--no-pager` 可禁用 pager，`GIT_PAGER` 控制分页程序，`GIT_EXTERNAL_DIFF` 会让 Git 调用外部 diff 程序。

建议所有 Git 命令统一加：

```
git --no-pager -c core.pager=cat -c color.ui=false ...
```

Rust 里设置环境变量：

```
use std::process::{Command, Stdio};

let output = Command::new(r"C:\Program Files\Git\cmd\git.exe")
    .args(["--no-pager", "-c", "core.pager=cat", "-c", "color.ui=false", "status", "--porcelain=v2", "-z"])
    .env("GIT_TERMINAL_PROMPT", "0")
    .env_remove("GIT_EXTERNAL_DIFF")
    .env_remove("GIT_DIFF_OPTS")
    .stdin(Stdio::null())
    .stdout(Stdio::piped())
    .stderr(Stdio::piped())
    .output()?;
```

关键点是：不要用 shell 启动 Git，直接指定 `git.exe`；stdout/stderr 要读完；stdin 设成 null；给命令加超时。

仓库位置也要避开 AnyShare 同步目录

如果你的代码仓库放在 AnyShare 同步盘、本地同步路径、缓存目录里，建议迁移到普通目录，例如：

```
C:\dev\your_repo
D:\work\your_repo
```

AnyShare 同步盘本身会做本地与云端双向同步、本地路径同步、自动下载/更新等操作。Git 会频繁访问 `.git\objects`、`.git\index`、工作区文件，和同步/HOOK/文件锁叠加后很容易放大卡顿或死锁概率。

如果要向管理员/厂商提工单

把这些证据带上最有效：

```
procdump -ma <hang住的git_pid> C:\temp\git_hang.dmp
procdump -ma <sharetool_pid> C:\temp\sharetool_hang.dmp
```

ProcDump 是微软 Sysinternals 工具，可用于生成进程 dump，也支持 hung window/异常等场景。

你这个场景最直接的解决路径是：

```
先 Exclude Git 安装目录 + Rust 工具目录
↓
重启 SyncClientService
↓
ProcMon 确认不再对 git.exe / your_tool.exe 启动 sharetool --hooks
↓
如果仍挂，再把 [Global] Mode=1 验证
```

这样可以保留 AnyShare，不需要卸载。

参考链接：- 爱数 AnyShare 智能云存储： <https://www.aishu.cn/cn/intelligent-cloud-storage> -
爱数博客： <https://www.aishu.cn/cn/blog/412> - Microsoft SetWindowsHookExW 文档： <https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-setwindowshookexw> -
Git 文档： <https://git-scm.com/docs/git> - AnyShare 文档： https://docs.aishu.cn/help/anyshare-enterprise-60/4258_3047 - Microsoft Sysinternals ProcDump： <https://learn.microsoft.com/en-us/sysinternals/downloads/procdump>

用户

帮我写个 powershell 脚本修改 Hookinfo.ini

助手

下面脚本保存为 `Update-Hookinfo.ps1`，用**管理员 PowerShell** 执行。

```
# Update-Hookinfo.ps1
# 用途: 修改 AnyShare Hookinfo.ini, 给 Git / 自己的程序增加 HOOK 排除项
# 建议: 右键 PowerShell -> 以管理员身份运行

param(
    # 不填则自动搜索 Hookinfo.ini
    [string]$IniPath = "",

    # 需要排除注入的目录, 按你的实际路径修改
    [string[]]$ExcludePaths = @(
        "C:\Program Files\Git\",
        "C:\dev\your-tool\"
    ),

    # 需要排除注入的进程名, 按你的实际程序名修改
    [string[]]$ExcludeProcesses = @(
        "git.exe",
        "ssh.exe",
        "git-remote-https.exe",
        "git-credential-manager.exe",
```

```

        "git-lfs.exe",
        "your_tool.exe"
    ),

    # 0 = 默认模式, 配合 Exclude 使用
    # 1 = 最小化注入, 影响范围更大, 只有 Exclude 无效时再用
    [ValidateSet(0, 1)]
    [int]$Mode = 0,

    # 修改后重启 AnyShare 同步服务
    [switch]$RestartService
)

function Assert-Admin {
    $identity = [Security.Principal.WindowsIdentity]::GetCurrent()
    $principal = New-Object Security.Principal.WindowsPrincipal($identity)
    $adminRole = [Security.Principal.WindowsBuiltInRole]::Administrator

    if (-not $principal.IsInRole($adminRole)) {
        throw "请使用管理员权限运行 PowerShell。"
    }
}

function Find-HookinfoIni {
    $candidates = @(
        "C:\Program Files (x86)\AISHU\Sync\Hookinfo.ini",
        "C:\Program Files\AISHU\Sync\Hookinfo.ini"
    )

    foreach ($path in $candidates) {
        if (Test-Path $path) {
            return $path
        }
    }

    $roots = @(
        "C:\Program Files (x86)\AISHU",
        "C:\Program Files\AISHU"
    )

    foreach ($root in $roots) {
        if (Test-Path $root) {
            $found = Get-ChildItem -Path $root -Filter "Hookinfo.ini" -Recurse
            -ErrorAction SilentlyContinue |
                Select-Object -First 1

            if ($found) {
                return $found.FullName
            }
        }
    }

    throw "没有找到 Hookinfo.ini. 请手动传入 -IniPath 参数。"
}

function Get-SectionRange {
    param(
        [System.Collections.Generic.List[string]]$Lines,
        [string]$Section
    )

```

```

$start = -1
$pattern = "^\\s*\\$([regex]::Escape($Section))\\s*$"

for ($i = 0; $i -lt $Lines.Count; $i++) {
    if ($Lines[$i] -match $pattern) {
        $start = $i
        break
    }
}

if ($start -eq -1) {
    return [pscustomobject]@{
        Exists = $false
        Start  = -1
        End    = -1
    }
}

$end = $Lines.Count - 1

for ($j = $start + 1; $j -lt $Lines.Count; $j++) {
    if ($Lines[$j] -match "\\s*\\$([regex]::Escape($Section))\\s*$") {
        $end = $j - 1
        break
    }
}

return [pscustomobject]@{
    Exists = $true
    Start  = $start
    End    = $end
}
}

function Ensure-Section {
    param(
        [System.Collections.Generic.List[string]]$Lines,
        [string]$Section
    )

    $range = Get-SectionRange -Lines $Lines -Section $Section

    if (-not $range.Exists) {
        if ($Lines.Count -gt 0 -and $Lines[$Lines.Count - 1].Trim() -ne "") {
            [void]$Lines.Add("")
        }

        [void]$Lines.Add("$Section")
    }
}

function Set-IniValue {
    param(
        [System.Collections.Generic.List[string]]$Lines,
        [string]$Section,
        [string]$Key,
        [string]$Value
    )

    Ensure-Section -Lines $Lines -Section $Section
    $range = Get-SectionRange -Lines $Lines -Section $Section

```

```

$keyPattern = "^s*$([regex]::Escape($Key))\s*"

for ($i = $range.Start + 1; $i -le $range.End; $i++) {
    if ($Lines[$i] -match $keyPattern) {
        $Lines[$i] = "$Key=$Value"
        return
    }
}

$insertIndex = $range.End + 1
$Lines.Insert($insertIndex, "$Key=$Value")
}

function Normalize-IniValue {
    param([string]$Value)

    return $Value.Trim().Trim('"').Trim("'").ToLowerInvariant()
}

function Add-ExcludeItems {
    param(
        [System.Collections.Generic.List[string]]$Lines,
        [string[]]$Paths,
        [string[]]$Processes
    )

    Ensure-Section -Lines $Lines -Section "Exclude"
    $range = Get-SectionRange -Lines $Lines -Section "Exclude"

    $existingPaths = @{}
    $existingProcesses = @{}
    $maxPathIndex = -1
    $maxProcessIndex = -1

    for ($i = $range.Start + 1; $i -le $range.End; $i++) {
        $line = $Lines[$i]

        if ($line -match "^s*Path(\d+)\s*=\s*(.+?)\s*$") {
            $index = [int]$matches[1]
            $value = Normalize-IniValue $matches[2]
            $existingPaths[$value] = $true

            if ($index -gt $maxPathIndex) {
                $maxPathIndex = $index
            }
        }

        if ($line -match "^s*Process(\d+)\s*=\s*(.+?)\s*$") {
            $index = [int]$matches[1]
            $value = Normalize-IniValue $matches[2]
            $existingProcesses[$value] = $true

            if ($index -gt $maxProcessIndex) {
                $maxProcessIndex = $index
            }
        }
    }

    $insertIndex = $range.End + 1

```

```

foreach ($path in $Paths) {
    if ([string]::IsNullOrEmpty($path)) {
        continue
    }

    $normalized = Normalize-IniValue $path

    if (-not $existingPaths.ContainsKey($normalized)) {
        $maxPathIndex++
        $Lines.Insert($insertIndex, "Path$maxPathIndex=$path")
        $insertIndex++
        $existingPaths[$normalized] = $true
        Write-Host "添加 Path$maxPathIndex=$path"
    } else {
        Write-Host "已存在 Path=$path"
    }
}

foreach ($process in $Processes) {
    if ([string]::IsNullOrEmpty($process)) {
        continue
    }

    $normalized = Normalize-IniValue $process

    if (-not $existingProcesses.ContainsKey($normalized)) {
        $maxProcessIndex++
        $Lines.Insert($insertIndex, "Process$maxProcessIndex=$process")
        $insertIndex++
        $existingProcesses[$normalized] = $true
        Write-Host "添加 Process$maxProcessIndex=$process"
    } else {
        Write-Host "已存在 Process=$process"
    }
}
}

Assert-Admin

if ([string]::IsNullOrEmpty($IniPath)) {
    $IniPath = Find-HookinfoIni
}

if (-not (Test-Path $IniPath)) {
    throw "Hookinfo.ini 不存在: $IniPath"
}

Write-Host "Hookinfo.ini: $IniPath"

if ($RestartService) {
    Write-Host "正在停止 SyncClientService..."
    Stop-Service -Name "SyncClientService" -Force -ErrorAction SilentlyContinue
}

$timestamp = Get-Date -Format "yyyyMMdd_HH:mm:ss"
$backupPath = "$IniPath.bak.$timestamp"
Copy-Item -Path $IniPath -Destination $backupPath -Force
Write-Host "已备份到: $backupPath"

$encoding = [System.Text.Encoding]::Default
$content = [System.IO.File]::ReadAllLines($IniPath, $encoding)

```



```

$lines = New-Object System.Collections.Generic.List[string]
$lines.AddRange([string[]]$content)

Set-IniValue -Lines $lines -Section "Global" -Key "Mode" -Value $Mode
Add-ExcludeItems -Lines $lines -Paths $ExcludePaths -Processes $ExcludeProcesses

[System.IO.File]::WriteAllLines($IniPath, $lines.ToArray(), $encoding)

Write-Host ""
Write-Host "修改完成。当前关键配置如下："
Write-Host "-----"

$lines | Where-Object {
    $_ -match "^\\s*[Global]\\s*$" `
    -or $_ -match "^\\s*[Exclude]\\s*$" `
    -or $_ -match "^\\s*Mode\\s*=" `
    -or $_ -match "^\\s*Path\\d+\\s*=" `
    -or $_ -match "^\\s*Process\\d+\\s*="
} | ForEach-Object {
    Write-Host $_
}

Write-Host "-----"

if ($RestartService) {
    Write-Host "正在启动 SyncClientService..."
    Start-Service -Name "SyncClientService" -ErrorAction SilentlyContinue
    Write-Host "服务已尝试重启。"
} else {
    Write-Host "建议重启 AnyShare 客户端或执行："
    Write-Host "Stop-Service SyncClientService -Force"
    Write-Host "Start-Service SyncClientService"
}

```

执行示例：

```

Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\Update-Hookinfo.ps1 -RestartService

```

指定你自己的 Rust 程序目录和进程名：

```

.\Update-Hookinfo.ps1 `
-ExcludePaths "C:\Program Files\Git\","D:\work\git-ai" `
-ExcludeProcesses "git.exe", "ssh.exe", "git-remote-https.exe", "git-ai.exe" `
-RestartService

```

如果 `Exclude` 之后仍然挂起，再临时测试最小化注入：

```

.\Update-Hookinfo.ps1 -Mode 1 -RestartService

```

建议先用 `Mode=0 + Exclude`，不要一上来就 `Mode=1`，因为 `Mode=1` 可能影响 AnyShare 同步盘的一些拦截下载能力。